

NPTEL Lecture Translation Pipeline

End-to-End Automatic Dubbing and Lip Synchronisation
for Indian-Language Education

Bachelor of Technology Project Report
Department of Computer Science and Engineering
IIIT Hyderabad

Aryan Chugh (2023101016) Aditya Gaur (2023101052)

Advisor: Prof. Vasudev Varma

May 8, 2026

Abstract

This report presents an end-to-end automatic video dubbing pipeline designed to translate English NPTEL lecture videos into Hindi, Telugu, and Odia — complete with dubbed audio, soft subtitles, voice cloning, and lip synchronisation. The system chains seven distinct machine-learning and signal-processing stages: audio extraction, music separation (Demucs), speech-to-text (faster-Whisper), neural machine translation (GCP Cloud Translate / Gemini), text-to-speech (Google Cloud TTS Neural2), tone-colour voice cloning (OpenVoice v2), temporal alignment (Rubber-Band), and facial lip synchronisation (Wav2Lip). Each component was evaluated against alternatives; the design decisions and performance trade-offs are documented here. Sample outputs are available at: [Google Drive \(sample outputs\)](#). The full source code is available at: [GitHub \(codebase\)](#).

0 Contents

1	Introduction	3
1.1	Design Goals	3
2	System Overview	3
2.1	User Interface	3
2.2	High-Level Pipeline Architecture	4
2.3	Output Artefacts	5
3	Stage 1: Audio Extraction	6
4	Stage 1b: Music Separation (Demucs)	6
4.1	Why Demucs	6
4.2	Auto-fallback	7
5	Stage 2: Speech-to-Text (Whisper)	7
5.1	Engine Comparison	7
5.2	VAD Integration	8
5.3	Duration-balanced chunking	8
6	Stage 3: Translation	8
6.1	Engine Comparison	8
6.2	Technical Term Preservation (Code-Mixing)	8
7	Stage 4: Subtitle Generation	9
8	Stage 5: Text-to-Speech Synthesis	9
8.1	TTS Engine Comparison	9
8.2	Duration-Aware SSML Rate Control	10
9	Stage 5b: Voice Cloning (OpenVoice v2)	10
9.1	How OpenVoice v2 Works	10
9.2	Reference Clip Extraction	10
9.3	Per-Segment Conversion	11
9.4	Quality Verification	11
10	Stage 6: Temporal Alignment	11
10.1	Rubber Band vs. ffmpeg atempo	11
10.2	Bounds and Padding	12
10.3	Pitch Shift Post-processing	12
11	Stage 6c: Music Remix	12
12	Stage 7: Lip Synchronisation (Wav2Lip)	12
12.1	Checkpoint Selection	12
12.2	Segment-Aware Processing	12
12.3	RAM-Constrained Frame Processing	13
12.4	Face Detection Robustness	13
12.5	VRAM Budget (4 GB GPU)	13

13 Stage 8: MKV Multiplexing	13
14 Caching and Resumability	14
15 Voice Analysis and Filler Preservation	14
15.1 Voice Analysis	14
15.2 Filler Preservation	14
16 Sample Outputs	15
17 Limitations and Future Work	15

1 Introduction

India's National Programme on Technology Enhanced Learning (NPTEL) provides free video lectures from IITs and IIMs to millions of students. However, the content is delivered exclusively in English, creating a significant access barrier for students whose first language is Hindi, Telugu, or Odia.

Manual dubbing of a one-hour lecture requires dozens of person-hours and professional voice actors for every target language. The goal of this project is to automate this process entirely: given a raw English lecture video, produce fully dubbed output videos for multiple Indian languages without any human intervention.

The pipeline is implemented in Python and exposed through a browser-based Streamlit interface. All heavy computation (Whisper, OpenVoice, Wav2Lip) runs on the local GPU. Cloud API calls (GCP Translation, GCP TTS, Sarvam, Gemini) are used for tasks where local models had lower quality.

1.1 Design Goals

- G1. Fully automated** — no human intervention from raw video to dubbed output.
- G2. Temporally aligned** — dubbed speech must not overlap with subsequent English sentences; silence gaps must be preserved.
- G3. Voice-consistent** — dubbed audio should sound like the original lecturer, not a generic TTS robot.
- G4. Resource-constrained** — must run on a consumer GPU with 4 GB VRAM (NVIDIA GTX 1650 Ti class).
- G5. Resumable** — intermediate results are cached; a crash at any stage does not require restarting from scratch.

2 System Overview

2.1 User Interface

The pipeline is exposed through a Streamlit web application (`app.py`). Figure 1 shows the main interface, which allows the user to upload a lecture video, configure all pipeline stages through collapsible panels in the sidebar, and monitor progress through real-time logs streamed to the browser.

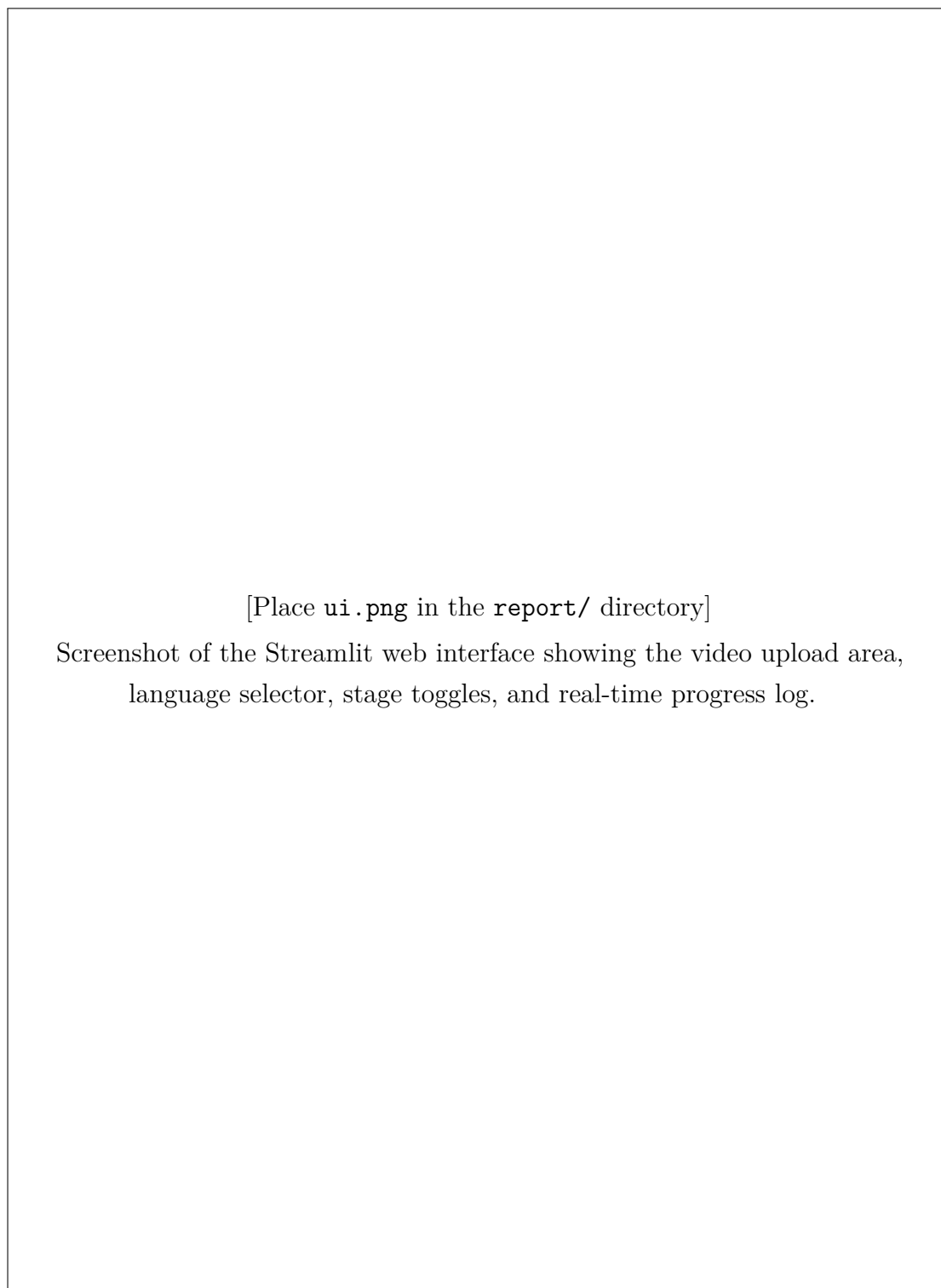


Figure 1: Streamlit UI — users upload a video and configure each pipeline stage from the left sidebar before clicking Run.

2.2 High-Level Pipeline Architecture

Figure 2 shows all stages and their data flow.

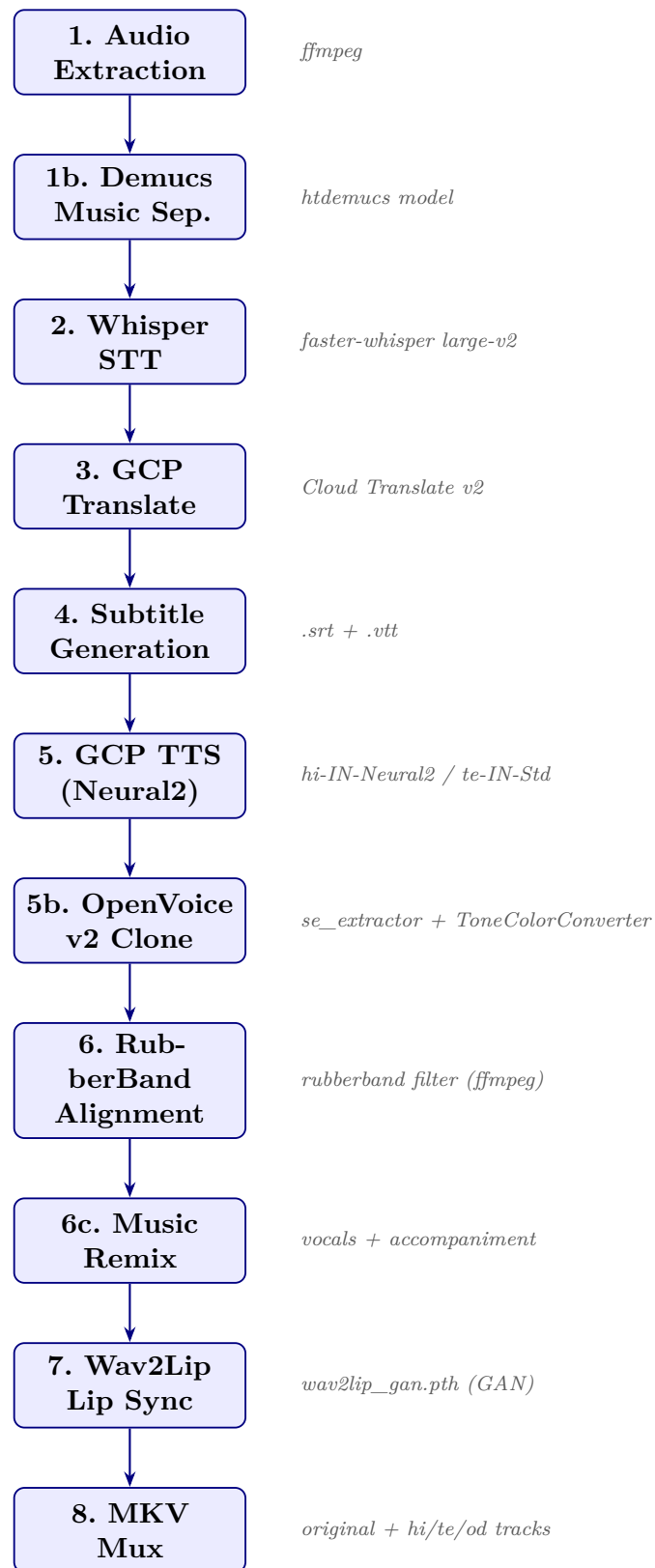


Figure 2: Pipeline stages and the technologies used at each step. Optional stages (music separation, voice cloning, lip sync) are individually toggleable from the UI.

2.3 Output Artefacts

For each input video the pipeline produces:

File	Description
*_dubbed.mkv	Multi-track MKV: original video, all audio and subtitle tracks
*_lipsync_hi.mkv	Lip-synced video for Hindi
*_hi_aligned.mp3	Dubbed Hindi audio aligned to original timing
*_hi_mixed.mp3	Hindi audio with original background music re-added
*_hi.srt / .vtt	Hindi subtitle file
lipsync_debug/ .cache/	Annotated frames showing face-detection bounding boxes JSON snapshots for resuming interrupted runs

Table 1: Output artefacts generated for a single input video (Hindi shown; analogous for Telugu and Odia).

3 Stage 1: Audio Extraction

Tool: ffmpeg

The first stage strips the audio channel from the input video and writes a 16 kHz mono WAV file. This format is the industry standard for speech recognition APIs and avoids downstream re-sampling artefacts.

```
ffmpeg -i lecture.mp4 -ar 16000 -ac 1 lecture.wav
% 16 kHz mono WAV for downstream STT
```

All subsequent stages operate on this WAV file, never the original video, until the final mux step.

4 Stage 1b: Music Separation (Demucs)

Tool: Meta Demucs (htdemucs model)

Many NPTEL lectures include opening and closing background music, title-card transitions, or soft ambient audio. If this is passed directly to the STT engine, word error rate increases significantly. Demucs separates the audio into two stems:

- **Vocals** (vocals.wav) — isolated lecturer speech fed to Whisper.
- **Accompaniment** (no_vocals.wav) — background music held aside and re-mixed at the end.

4.1 Why Demucs

We evaluated three source-separation approaches:

Approach	Pros	Cons
Simple high-pass filter	No dependencies	Cannot separate vocals; damages speech
Spleeter (Deezer, 2019)	Fast, simple API	Lower separation quality, stale (Python 3.8 only)
Demucs htdemucs	State-of-art separation, single pip install, CPU fallback	Slightly slower (30 s/min of audio)

Table 2: Music separation alternatives evaluated.

Demucs htdemucs was selected as the default. It is the only option that reliably preserves voice quality while cleanly removing background music in NPTEL videos. The model (80 MB) is downloaded automatically on first use.

4.2 Auto-fallback

If the STT stage returns “no speech detected” on the original audio (e.g. music-heavy intro), the pipeline automatically triggers Demucs even when the user did not enable it, then retries transcription on the clean vocals stem.

5 Stage 2: Speech-to-Text (Whisper)

Tool: faster-whisper (CTranslate2 backend)

Default model: large-v2

5.1 Engine Comparison

Three STT backends were integrated and evaluated:

Engine	Timestamp quality	Accuracy	Cost
Sarvam AI STT	Sentence-level only	Good for Indian-accented English	API (paid)
Google Gemini	Sentence-level only	Very high; hallucination risk on music	API (paid)
Whisper large-v2	Word-level	Highest; native English	Free (local)

Table 3: STT engine comparison. Whisper is the default.

Whisper was chosen as the default for two critical reasons:

1. **Word-level timestamps** — Whisper’s cross-attention alignment produces per-word timestamps accurate to ~ 100 ms. Gemini and Sarvam return only sentence-level start/end times, requiring heuristic interpolation for sub-sentence gaps.
2. **Free and offline** — no API cost; runs entirely on the local GPU.

5.2 VAD Integration

Whisper is run with the built-in Voice Activity Detection (VAD) filter (`silero-vad` backend). Parameters:

- `min_speech_duration_ms` = 500
- `speech_pad_ms` = 200
- `threshold` = 0.5

This suppresses hallucination on silent sections and reduces transcription time by skipping non-speech regions.

5.3 Duration-balanced chunking

The word-level output is merged into sentence-level chunks using a duration-balanced algorithm. Each chunk is constrained to [5s, 13s] of original audio, with a target of 9s. This ensures TTS segments are never so short that time-stretching would be extreme, nor so long that a single segment spans multiple slides.

6 Stage 3: Translation

Default: Google Cloud Translation v2 (REST API)

Supported languages: Hindi (`hi`), Telugu (`te`), Odia (`od`)

6.1 Engine Comparison

Engine	Quality	Notes
Sarvam Translate	High for Indian languages	Limited context window; per-segment calls
Google Gemini	High; context-aware	Risk of ignoring glossary; slower batching
GCP Cloud Translate	High; fast batch	Reliable, 100-segment batches, proven Indic quality

Table 4: Translation engine comparison. GCP Cloud Translate is the default.

GCP Cloud Translate was selected because it provides consistent quality at high throughput (batches of 100 segments per API call) and has well-documented Indic language support. The pipeline uses `languageCode`: `hi`, `te`, or `or` (ISO 639-1 for Odia).

6.2 Technical Term Preservation (Code-Mixing)

NPTEL lectures contain a high density of English technical terms (“neural network”, “gradient descent”, “CUDA”, etc.) that must *not* be translated — they should appear in English in the target-language output, because equivalent terms are not standard in Hindi or Telugu pedagogy.

The pipeline implements a two-tier glossary:

1. **Static glossary** — a curated list of ~ 200 computer science, mathematics, and electronics terms hardcoded in `glossary.py`.
2. **Dynamic glossary** — Gemini analyses the actual transcript and extracts domain-specific terms unique to that lecture (e.g. “backpropagation”, “tokenisation”).

Terms from the combined glossary are injected into the translation prompt as explicit do-not-translate constraints. A post-processing pass verifies preservation and logs any missing terms.

7 Stage 4: Subtitle Generation

Formats: `.srt` and `.vtt` (Web Video Text Tracks)

Subtitle files are generated for all languages (English + all target languages) directly from the segment timestamps produced by Whisper (English) and propagated through translation (target languages).

Each subtitle line is word-wrapped at 42 characters per line and truncated if the segment is shorter than the natural reading time. Characters-per-second thresholds were empirically tuned for Hindi Devanagari (which is visually denser than Latin script).

The subtitle tracks are muxed as *soft subtitles* into the final MKV container, meaning they can be toggled on/off independently in any media player (VLC, mpv, PotPlayer) without burning them into the video.

8 Stage 5: Text-to-Speech Synthesis

Default: Google Cloud TTS (`gcptts_vc` engine = GCP Neural2 + OpenVoice v2)

Each translated segment is synthesised into a short audio clip (2–15 s). The output is one audio file per segment, e.g. `seg_0001.mp3`.

8.1 TTS Engine Comparison

Five engines were integrated and tested:

Engine	Voice quality	Indic language support	Requires
edge-tts (Microsoft)	Neural, good	Hindi, Telugu	Free, online
Sarvam TTS	Very good	Hi/Te/Od	Sarvam API key
Coqui XTTS v2 (local)	High; voice cloning built-in	Limited Indic	6 GB VRAM, slow
GCP TTS Standard	Moderate	Hi/Te	GCP key
GCP TTS Neural2	Highest naturalness	Hi/Te (Neural2/Standard)	GCP key

Table 5: TTS engine comparison. GCP Neural2 is the default for Hindi; Standard for Telugu (Neural2 unavailable for `te-IN`).

GCP TTS Neural2 was selected because it is the same WaveNet-based architecture used by YouTube’s auto-dubbing pipeline. The specific voice models are:

- Hindi: `hi-IN-Neural2-B` (male) / `hi-IN-Neural2-C` (female)
- Telugu: `te-IN-Standard-B` (male) / `te-IN-Standard-A` (female)
- Odia: GCP does not support `or-IN` → falls back to Sarvam TTS

8.2 Duration-Aware SSML Rate Control

Rather than synthesising at a fixed rate and always time-stretching afterwards, the pipeline first estimates the natural duration of the TTS output using a characters-per-second (CPS) model:

$$r = \text{clamp}\left(\frac{N_{\text{chars}}}{t_{\text{target}} \times \text{CPS}}, 0.65, 1.75\right) \quad (1)$$

where N_{chars} is the character count of the target-language text, t_{target} is the original segment duration in seconds, and $\text{CPS} = 12.0$ for Hindi Devanagari. The result is passed as an SSML `<prosody rate="...">` attribute. This two-pass approach reduces the time-stretch ratio needed in Stage 6, improving final audio naturalness.

9 Stage 5b: Voice Cloning (OpenVoice v2)

Tool: MyShell OpenVoice v2 ([GitHub](#))

Model size: ~300 MB, auto-downloaded from HuggingFace

Even after Neural2 TTS, the dubbed audio sounds like a generic GCP voice. OpenVoice v2 is a *language-agnostic* tone-colour transfer model that re-tones the TTS output to sound like the original lecturer.

9.1 How OpenVoice v2 Works

1. A **speaker encoder** (similar to GE2E/d-vector) maps any audio clip to a 256-dimensional embedding that captures timbre, resonance, and voice quality — independent of linguistic content.
2. The **ToneColorConverter** (a neural vocoder) re-synthesises the TTS audio using the source segment's *content* but the reference lecturer's *tone colour*.

9.2 Reference Clip Extraction

A high-quality 30-second voiced reference clip is extracted automatically from the lecture using Silero-VAD:

1. The lecture is divided into three temporal zones: 15–30%, 40–60%, and 65–80% through the video.
2. Silero-VAD detects voiced speech frames in each zone.
3. ~10s of clean speech per zone is concatenated → 30s total.
4. The result is resampled to 22 050 Hz mono (OpenVoice's required format) and bandpass-filtered (80–8000 Hz) to remove low-frequency rumble and high-frequency hiss.

Sampling across multiple zones rather than a single window ensures the embedding captures the lecturer’s voice across different speaking styles, energy levels, and topic contexts.

9.3 Per-Segment Conversion

Voice cloning is applied *per segment* (not on the full concatenated track) because:

- Each segment is short, focused, uninterrupted speech — ideal for the VAD inside the speaker encoder.
- The reference speaker embedding is computed once and cached in memory for the full run.
- The source embedding (per TTS segment) is computed fresh and immediately freed, keeping VRAM usage bounded.

Segments shorter than 0.5 s are skipped (insufficient voiced frames for reliable embedding extraction).

9.4 Quality Verification

After conversion, **Resemblyzer** computes the cosine similarity between the converted output and the reference embedding. This metric is logged for every segment; segments with similarity below 0.65 are flagged so the user can inspect them.

10 Stage 6: Temporal Alignment

Tool: RubberBand via `ffmpeg rubberband` audio filter

Translated speech is intrinsically longer than English in most Indian languages (Hindi tends to be 15–25% longer per sentence). The aligner time-stretches each TTS clip so it fits exactly within the original segment’s time window.

10.1 Rubber Band vs. `ffmpeg atempo`

Method	Pros	Cons
<code>ffmpeg atempo</code>	Simple; built-in	Phase-vocoder artefacts on speech; degrades formants
RubberBand	Phase-vocoder; pitch + formant preserving	Requires <code>librubberband</code> package

Table 6: Time-stretching method comparison. RubberBand is the default.

RubberBand’s `formant=preserved` option maintains vowel quality even at extreme stretch ratios, which is critical for Indic speech where vowel length is phonemically distinctive.

10.2 Bounds and Padding

The tempo ratio is clamped to $[0.55, 1.60]$. When the TTS audio is so long that even a $1.6\times$ speedup cannot fit it in the window, it is hard-truncated. When it is shorter than the window even after maximum slow-down, silence padding is inserted.

10.3 Pitch Shift Post-processing

After alignment, a separate pitch-shift step (via `ffmpeg rubberband`) adjusts the dubbed audio to match the original lecturer’s measured average pitch (extracted from the voice analysis stage). This prevents the jarring sensation of a lecturer who suddenly changes pitch between languages.

11 Stage 6c: Music Remix

The accompaniment stem saved by Demucs is mixed back with the aligned dubbed audio using `ffmpeg amix`:

```
ffmpeg -i dubbed_vocals.mp3 -i accompaniment.wav \
  -filter_complex "[0:a][1:a]amix=inputs=2:duration=first:weights
    =1_0.18" \
  output_mixed.mp3
```

The accompaniment weight (0.18) is calibrated so background music is audible but does not overpower speech. A loudness check is run first: if the accompaniment energy is near $-\infty$ dB (silence), the mix step is skipped to avoid re-encoding the audio needlessly.

12 Stage 7: Lip Synchronisation (Wav2Lip)

Tool: Wav2Lip (Prajwal et al., 2020) with GAN checkpoint (`wav2lip_gan.pth`, 436 MB)

Wav2Lip is a Generative Adversarial Network that animates a speaker’s mouth in video to match a provided audio track. It re-renders only the lower-face region in each frame, blending seamlessly with the unchanged upper face and background.

12.1 Checkpoint Selection

Checkpoint	Pros	Cons
<code>wav2lip.pth</code>	Best sync accuracy	Slight blurriness in lower face
<code>wav2lip_gan.pth</code>	Higher visual quality (GAN-sharpened)	Marginally lower sync precision

Table 7: Wav2Lip checkpoint comparison. The GAN checkpoint is the default.

12.2 Segment-Aware Processing

NPTEL lecture videos frequently switch between a camera shot of the lecturer and a slide view (no face). Running Wav2Lip on the full video fails immediately when it encounters a slide frame.

The pipeline implements segment-aware processing:

1. **Face detection scan:** the video is sampled at 0.5 s intervals using the S3FD face detector. Results are merged into segments of type `has_face=True/False`.
2. **Face segments** are passed to Wav2Lip.
3. **Slide segments** use a simple ffmpeg audio-swap (no face animation needed).
4. All processed clips are concatenated with ffmpeg.

12.3 RAM-Constrained Frame Processing

Wav2Lip’s `inference.py` loads the *entire* video into a Python list before processing. For a 2-minute 1080p face segment this requires ≈ 9.5 GB of RAM. Two optimisations were implemented:

1. **Pre-scale to 720p** — since Wav2Lip internally resizes every detected face crop to 96×96 before inference, there is no quality gain from feeding 1080p input. ffmpeg scales the clip to ≤ 720 px height before calling `inference.py`.
2. **≤ 30 s sub-chunks** — clips longer than 30 s are split into sub-chunks of exactly 30 s, processed independently (each requiring ≈ 2 GB RAM), then concatenated.

12.4 Face Detection Robustness

The original Wav2Lip code raises `ValueError: Face not detected` as soon as any single frame has no detection. The patched version:

1. Collects all per-frame detection results first.
2. Forward-fills missing detections from the nearest preceding valid detection.
3. Backward-fills any remaining leading Nones.
4. Only raises an error if *every* frame lacks a face.

12.5 VRAM Budget (4 GB GPU)

Parameter	Value
<code>resize_factor</code>	1 (full resolution input after 720p pre-scale)
<code>wav2lip_batch_size</code>	8 frames per forward pass
<code>face_det_batch_size</code>	1 (one frame at a time to avoid RAM spike)
<code>PYTORCH_CUDA_ALLOC_CONF</code>	<code>expandable_segments:True</code>

Table 8: Wav2Lip parameters tuned for 4 GB VRAM.

13 Stage 8: MKV Multiplexing

Tool: ffmpeg

The final step assembles all streams into a single `.mkv` Matroska container:

- Video stream: original (or lip-synced MP4 for per-language outputs).
- Audio track 0: English original.
- Audio tracks 1– n : Hindi, Telugu, Odia dubbed tracks.
- Subtitle tracks: one per language (.srt muxed as ASS).

Using a soft multi-track container means one MKV file can be played in VLC with any language audio and any subtitle combination selected at runtime, without re-encoding.

14 Caching and Resumability

Every major stage writes its result as a JSON file to `output/<video>/cache/` before proceeding. On the next run, if a cache file is found for a stage it is loaded and that stage is skipped. This means:

- A crash after Whisper (the slowest step) does not require re-transcribing.
- Changing the TTS engine only reruns Stage 5 onwards.
- Different target languages can be added without re-running STT or translation for languages already completed.

15 Voice Analysis and Filler Preservation

15.1 Voice Analysis

Before TTS synthesis, `voice_analyzer.py` extracts from the original audio:

- Speaking rate (words per second)
- Average pitch / fundamental frequency (F0)
- Pitch range (min/max Hz)
- Average loudness (dBFS)
- Inferred gender (from F0: < 165 Hz $\rightarrow$ male; > 165 Hz $\rightarrow$ female)

The pitch measurement feeds the post-alignment pitch-shift step. The speaking rate informs SSML rate estimation in the TTS stage.

15.2 Filler Preservation

Lecturers naturally use hesitation markers (**um**, **uh**, **ah**, **hmm**, etc.) that contribute to natural speech rhythm. The pipeline detects these in the Whisper word-level output and stores their timing. The TTS stage can optionally re-inject them as pauses in the dubbed audio using SSML `<break>` tags, preserving the lecturer's natural rhythm in the dubbed output.

16 Sample Outputs

Sample output videos demonstrating the full pipeline on NPTEL lecture clips are publicly available at:

https://drive.google.com/drive/folders/1_JSBJKzM6jmwPKTSqcICVxC-7uuiQ1k?usp=drive_link

The complete source code for this project is hosted on GitHub:

https://github.com/Aditya-Gaur26/multilingual_video_translation

17 Limitations and Future Work

- L1. Odia TTS quality** — GCP does not support **or-IN** Neural2 voices. The Sarvam fallback is lower quality than Hindi/Telugu. Future work: fine-tune a local VITS model on Odia data.
- L2. Lip sync on partial faces** — Wav2Lip degrades on side-profile shots or when the lecturer turns away. The bounding-box interpolation patch mitigates but does not eliminate this.
- L3. Slide-text translation** — text embedded in slide images (equations, code snippets) is not translated. OCR + overlay rendering would be required.
- L4. Multi-speaker videos** — the current pipeline assumes a single speaker. Panel discussions or Q&A segments with multiple speakers would require speaker diarisation.
- L5. Gender-adaptive TTS** — voice gender is currently hardcoded to male. Adding automatic gender detection from the voice profile would improve naturalness for female lecturers.